

SKILL WEEK-7

NAME:CH.SAI VIHAMSH

ROLL:2520090090

SESSION-13

Task1: To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

```
saiviamsh@Varshini-PC:~$ cd ~/OSSP/skill
saiviamsh@Varshini-PC:~/OSSP/skill$ mkdir -p week7/task1
saiviamsh@Varshini-PC:~/OSSP/skill$ cd week7/task1
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ nano input_redirect.c
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ echo "Hello from input r
edirection" > input.txt
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ cat input.txt
Hello from input redirection
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ gcc input_redirect.c -o
T1
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ ls -l
total 28
-rwxr-xr-x 1 madhulika madhulika 16824 Sep 17 13:37 T1
-rw-r--r-- 1 madhulika madhulika   29 Sep 17 13:36 input.txt
-rw-r--r-- 1 madhulika madhulika  2127 Sep 17 13:36 input_redirect.c
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ ./T1
=== Skill Week 7 - Task 1 ===
Input Redirection

Enter command: cat < input.txt
Input file: input.txt
Input file opened successfully.
Hello from input redirection

Input stream restored to normal.
Input redirection test completed.
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ ./T1
=== Skill Week 7 - Task 1 ===
Input Redirection

Enter command: wc -w < input.txt
Input file: input.txt
Input file opened successfully.
4

Input stream restored to normal.
Input redirection test completed.
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_COMMAND 200

int main()
{
    char command[MAX_COMMAND];
    char filename[MAX_COMMAND];
    char *args[20];
    char *token;
    int fd;
    int count;
    pid_t pid;

    printf("=== Skill Week 7 - Task 1 ===\n");
    printf("Input Redirection\n\n");

    printf("Enter command: ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
    {
        printf("Failed to read command.\n");
        return 1;
    }

    command[strcspn(command, "\n")] = '\0';

    char *redirect = strstr(command, "<");

    if (redirect == NULL)
    {

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```

        pid = fork();

        if (pid < 0)
        {
            perror("fork");
            close(fd);
            return 1;
        }

        if (pid == 0)
        {
            if (dup2(fd, STDIN_FILENO) == -1)
            {
                perror("dup2");
                close(fd);
                exit(1);
            }

            close(fd);

            execvp(args[0], args);

            perror("execvp");
            exit(1);
        }

        close(fd);

        waitpid(pid, NULL, 0);

        printf("\nInput stream restored to normal.\n");
        printf("Input redirection test completed.\n");

        return 0;
    }
}

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

Observation: The program successfully parsed the input redirection symbol <, extracted the input file name, and opened the specified file in read-only mode. The file descriptor was redirected to the standard input of a child process using dup2(), allowing commands such as cat and wc to read data directly from the file. The program correctly handled missing input files by displaying an appropriate error message and also closed the file descriptors after use. The successful execution of different commands confirmed that input data was correctly passed through the redirected standard input stream.

Task2: To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

```
saiviamsh@Varshini-PC:~/OSSP/skill$ mkdir -p week7/task2
saiviamsh@Varshini-PC:~/OSSP/skill$ cd week7/task2
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ nano output_redirect.c
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ ^C
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ gcc output_redirect.c -o
T2
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ ^C
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ ls -l
total 24
-rwxr-xr-x 1 madhulika madhulika 16824 Sep 17 13:42 T2
-rw-r--r-- 1 madhulika madhulika 2171 Sep 17 13:42 output_redirect.c
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ ./T2
=== Skill Week 7 - Task 2 ===
Output Redirection

Enter command: echo Hello World > output.txt
Output file: output.txt
Output file created/opened successfully.

Output stream restored to normal.
Output redirection test completed.
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ cat output.txt
Hello World
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ ls -l output.txt
-rw-r--r-- 1 madhulika madhulika 12 Sep 17 13:43 output.txt
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ ./T2
=== Skill Week 7 - Task 2 ===
Output Redirection

Enter command: echo Hello >
Error: Output file not specified.
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_COMMAND 200

int main()
{
    char command[MAX_COMMAND];
    char filename[MAX_COMMAND];
    char *args[20];
    char *token;
    int fd;
    int count;
    pid_t pid;

    printf("=== Skill Week 7 - Task 2 ===\n");
    printf("Output Redirection\n\n");

    printf("Enter command: ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
    {
        printf("Failed to read command.\n");
        return 1;
    }

    command[strcspn(command, "\n")] = '\0';

    char *redirect = strstr(command, ">");

    if (redirect == NULL)
    {

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

GNU nano 8.7.1 output_redirect.c *

```

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0)
    {
        if (dup2(fd, STDOUT_FILENO) == -1)
        {
            perror("dup2");
            close(fd);
            exit(1);
        }

        close(fd);

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }

    close(fd);

    waitpid(pid, NULL, 0);

    printf("\nOutput stream restored to normal.\n");
    printf("Output redirection test completed.\n");

    return 0;
}

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

Observation: The program successfully parsed the output redirection symbol >, extracted the output file name, and created or opened the specified file using appropriate file permissions. The standard output of commands was redirected to the output file using dup2(), and the parent process waited for the child process to complete before restoring normal output. The generated files were checked using cat and ls, confirming that the command output was correctly stored in the files. Existing files were also successfully overwritten using output redirection, while invalid redirection syntax and missing output filenames were handled appropriately.

SESSION-14

Task1: To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

```
saiviamsh@Varshini-PC:~/OSSP/skill$ cd week7/task1
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ nano append_redirect.c
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ gcc append_redirect.c -o
T1
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ ls -l
total 32
-rwxr-xr-x 1 madhulika madhulika 16824 Sep 17 13:52 T1
-rw-r--r-- 1 madhulika madhulika 2191 Sep 17 13:52 append_redirect.c
-rw-r--r-- 1 madhulika madhulika 29 Sep 17 13:36 input.txt
-rw-r--r-- 1 madhulika madhulika 2127 Sep 17 13:36 input_redirect.c
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ echo "First line" > appe
nd.txt
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ cat append.txt
First line
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ ./T1
=== Skill Week 7 - Task 1 ===
Append Output Redirection

Enter command: echo "Second line" >> append.txt
Output file: append.txt
File opened in append mode successfully.

Output stream restored to normal.
Append redirection test completed.
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ cat append.txt
First line
"Second line"
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ ./T1
=== Skill Week 7 - Task 1 ===
Append Output Redirection

Enter command: echo "Third line" >> append.txt
Output file: append.txt
File opened in append mode successfully.

Output stream restored to normal.
Append redirection test completed.
```

```

GNU nano 2.7.1 append_redirected.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_COMMAND 200

int main()
{
    char command[MAX_COMMAND];
    char filename[MAX_COMMAND];
    char *args[20];
    char *token;
    char *redirect;
    int fd;
    int count;
    pid_t pid;

    printf("=== Skill Week 7 - Task 1 ===\n");
    printf("Append Output Redirection\n\n");

    printf("Enter command: ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
    {
        printf("Failed to read command.\n");
        return 1;
    }

    command[strcspn(command, "\n")] = '\0';

    redirect = strstr(command, ">>");

    if (redirect == NULL)

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

```

pid = fork();

if (pid < 0)
{
    perror("fork");
    close(fd);
    return 1;
}

if (pid == 0)
{
    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        exit(1);
    }

    close(fd);

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

close(fd);

waitpid(pid, NULL, 0);

printf("\nOutput stream restored to normal.\n");
printf("Append redirection test completed.\n");

return 0;
}

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

Observation: The program successfully parsed the append operator >> and opened the specified output file using append mode. The standard output of the child process was redirected to the file using dup2(), while the parent process closed the file descriptor and synchronized with the child using waitpid(). Testing with multiple commands showed that new data was added to the end of the existing file without removing previously stored contents. The program also correctly handled missing output filenames and incorrect redirection operators, confirming that append operations were properly validated.

Task2: To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

```
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task1$ cd ~/OSSP/skill
saiviamsh@Varshini-PC:~/OSSP/skill$ mkdir -p week7/task2
saiviamsh@Varshini-PC:~/OSSP/skill$ cd week7/task2
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ nano stderr_redirect.c
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ gcc stderr_redirect.c -o
T2
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ ls -l
total 32
-rwxr-xr-x 1 madhulika madhulika 16824 Sep 17 14:05 T2
-rw-r--r-- 1 madhulika madhulika  12 Sep 17 13:43 output.txt
-rw-r--r-- 1 madhulika madhulika 2171 Sep 17 13:42 output_redirect.c
-rw-r--r-- 1 madhulika madhulika 2364 Sep 17 14:05 stderr_redirect.c
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ ls /does_not_exist 2> er
ror.txt
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ ./T2
=== Skill Week 7 - Task 2 ===
stderr Redirection

Enter command: ls /does_not_exist 2> error.txt
Error output file: error.txt
Error-output file created/opened successfully.

Child process exited with status: 2
stderr stream restored to normal.
stderr redirection test completed.
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ cat error.txt
ls: cannot access '/does_not_exist': No such file or directory
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ ls -l error.txt
-rw-r--r-- 1 madhulika madhulika 63 Sep 17 14:06 error.txt
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ ./T2
=== Skill Week 7 - Task 2 ===
stderr Redirection

Enter command: cat missing_file.txt 2> cat_error.txt
Error output file: cat_error.txt
Error-output file created/opened successfully.

Child process exited with status: 1
stderr stream restored to normal.
stderr redirection test completed.
saiviamsh@Varshini-PC:~/OSSP/skill/week7/task2$ cat cat_error.txt
```

```
GNU nano 2.7.1          stderr_redirect.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_COMMAND 200

int main()
{
    char command[MAX_COMMAND];
    char filename[MAX_COMMAND];
    char *args[20];
    char *token;
    char *redirect;
    int fd;
    int count;
    int status;
    pid_t pid;

    printf("=== Skill Week 7 - Task 2 ===\n");
    printf("stderr Redirection\n\n");

    printf("Enter command: ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
    {
        printf("Failed to read command.\n");
        return 1;
    }

    command[strcspn(command, "\n")] = '\0';
    redirect = strstr(command, "2>");

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
        close(fd);
        return 1;
    }

    if (pid == 0)
    {
        if (dup2(fd, STDERR_FILENO) == -1)
        {
            perror("dup2");
            close(fd);
            exit(1);
        }

        close(fd);

        execvp(args[0], args);

        perror("execvp");
        exit(1);
    }

    close(fd);

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("\nChild process exited with status: %d\n",
            WEXITSTATUS(status));
    }

    printf("stderr stream restored to normal.\n");
    printf("stderr redirection test completed.\n");

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```


Observation: The program successfully parsed the `2> stderr` redirection operator and redirected the standard error stream of the child process to the specified file using `dup2()`. Error messages generated by invalid commands or missing files were captured in the output files instead of being displayed on the terminal. The contents of the generated error files were verified using the `cat` command, while successful commands produced empty error files because no error output was generated. The program also handled missing error-output filenames and commands without the `2>` operator appropriately.